Python Cheatsheet

Teaching Week 01 to 12 | The R sheet carries the same sections in the same order.

This sheet holds the calls this course teaches, in the order it teaches them. The R cheatsheet carries the same nine sections with the same entries in the same positions, so a student who knows one language can find the other call by reading across. Every example works on `heart.csv`, the table Class 01 opens with, through its `Age`, `MaxHR`, `Cholesterol`, and `HeartDisease` columns. The exceptions are the few lines whose comment names a stand-in (`lookup`, `num_obs`, `num_pred`), which need an object of your own. R reaches the first four sections without loading anything, so every import below is stated where it is first needed and belongs at the top of your script.

1 Reading a Table

A lab starts by reading one file into one table.

```
import pandas as pd

# Read a comma-separated file into a data frame.
heart = pd.read_csv("heart.csv")

# State the separator and the decimal mark when they are not the defaults.
heart = pd.read_csv("heart.csv", sep=",", decimal=".")

# Read the first rows only, while you check that the file parses.
pd.read_csv("heart.csv", nrows=20)

# Treat the strings a source writes for "no value" as missing.
pd.read_csv("heart.csv", na_values=["", "NA", "-99"])
```

2 Inspecting It

Confirm the shape, the names, and the type of every column before computing a single statistic.

```
heart.shape                          # rows and columns
heart.columns                        # column names
heart.head(5)                        # the first five rows
heart.dtypes                         # the storage type of every column
heart.describe()                     # a per-column summary of the whole table
heart["HeartDisease"].value_counts()  # a count for every level of one column
```

3 Summary Statistics

Centre, spread, quantiles, and correlation, one call each.

```
heart["Age"].mean()                    # arithmetic mean
heart["Age"].median()                  # middle value
heart["Age"].std()                     # standard deviation, divisor n - 1
heart["Age"].var()                     # variance, the square of the above
heart["Age"].agg(["min", "max"])       # smallest and largest value
heart["Age"].quantile([0.25, 0.75])    # first and third quartile
heart["Age"].quantile(0.75) - heart["Age"].quantile(0.25)   # Q3 - Q1
heart[["Age", "MaxHR"]].corr()         # correlation matrix of two columns
```

4 Missing Values

Count what is missing before deciding what to do about it, and never let a default silently decide for you.

```
# heart.csv writes an unmeasured cholesterol as 0, so recode it before counting.
heart["Cholesterol"] = heart["Cholesterol"].mask(heart["Cholesterol"] == 0)
heart["Cholesterol"].isna().sum()  # how many values of one column are missing
heart.isna().sum()                 # the same count for every column
heart.index[heart["Cholesterol"].isna()]   # the positions of the missing ones
heart.notna().all(axis=1)          # True where a row has no missing value
clean = heart.dropna()             # drop every incomplete row
heart["Cholesterol"] = heart["Cholesterol"].fillna(
    heart["Cholesterol"].median())         # fill with the median instead
heart["Age"].mean()                # a statistic skips NA unless told not to
```

5 Plotting

The base call is for a quick look at your screen. The grammar call is for the figure the report cites. Unlike R, pyplot draws every call on the current figure, so each plot below opens its own with `plt.figure()`.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure()                                # a new figure for the next plot
plt.hist(heart["Age"])                      # base histogram, quick look
plt.figure()
plt.boxplot(heart["Age"])                   # base boxplot, quick look
plt.figure()
plt.scatter(heart["Age"], heart["MaxHR"])   # base scatter plot, quick look

plt.figure()
sns.histplot(heart, x="Age", binwidth=3)    # histogram with a width you chose

plt.figure()
sns.scatterplot(heart, x="Age", y="MaxHR",
                hue="HeartDisease", alpha=0.6)  # coloured by the outcome
plt.xlabel("Age")
plt.ylabel("Maximum heart rate")

plt.savefig("results/age-maxhr.png")        # write the last plot to a file
```

6 Transforming

Filter, select, derive, sort, group, and summarise, in one chain that reads top to bottom.

```
from sklearn.preprocessing import StandardScaler

(heart
 .query("Age >= 40")                          # keep the rows that qualify
 [["Age", "MaxHR", "HeartDisease"]]           # keep the columns you use
 .assign(band=lambda d: pd.cut(d["Age"], 4))  # add a derived column
 .sort_values("MaxHR", ascending=False)       # sort, largest first
 .groupby("HeartDisease")                     # form the groups
 .agg(n=("Age", "size"), mean_age=("Age", "mean")))  # one row per group

StandardScaler().fit_transform(heart[["Age", "MaxHR"]])   # centre and divide
pd.Categorical(heart["HeartDisease"])   # read a stored code as a category
# lookup stands in for a second table that shares an id column with heart.
heart.merge(lookup, on="id", how="left")  # add the columns of a second table
```

7 A Decision Tree

Fit, draw, prune, and predict. The predictors go in one frame and the outcome stays beside it.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

X = heart[["Age", "MaxHR"]]   # the predictors
y = heart["HeartDisease"]     # the outcome

fit = DecisionTreeClassifier(
    criterion="gini",         # DecisionTreeRegressor predicts a number instead
    min_samples_split=20,
    max_depth=2,
    random_state=3141,
).fit(X, y)

plot_tree(fit)                          # draw the fitted tree
fit.cost_complexity_pruning_path(X, y)  # complexity table, one row per step
# ccp_alpha is the unscaled alpha on Gini impurity, not rpart's cp, which is
# alpha / R(root) on misclassification risk, so the two values differ.
# Pick a ccp_alpha between two steps of the path and refit.
fit = DecisionTreeClassifier(criterion="gini", min_samples_split=20,
                             max_depth=2, ccp_alpha=0.03,
                             random_state=3141).fit(X, y)  # cut it back
fit.predict(X)                          # the predicted class of every row
```

8 K-Means Clustering

Scale first, because the algorithm measures distance and an unscaled column decides the answer on its own. Fix the seed so the run repeats.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_samples, silhouette_score

Z = StandardScaler().fit_transform(heart[["Age", "MaxHR"]])
km = KMeans(n_clusters=3, n_init=25, max_iter=100,
            random_state=3141).fit(Z)

km.labels_                          # the cluster label of every row
km.cluster_centers_                 # the k centres, in scaled units
km.inertia_                         # total within-cluster sum of squares
silhouette_samples(Z, km.labels_)  # the silhouette width of every row
silhouette_score(Z, km.labels_)    # the average silhouette width
```

9 Evaluating a Model

Split the data, score the rows the model has not seen, and report the metric the outcome type calls for.

```
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, r2_score,
                             root_mean_squared_error)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, train_size=0.7, random_state=3141)   # 70 percent to fit on

fit = DecisionTreeClassifier(random_state=3141).fit(X_train, y_train)
pred = fit.predict(X_test)

confusion_matrix(y_test, pred)        # observed against predicted
accuracy_score(y_test, pred)          # accuracy, the share it got right
classification_report(y_test, pred)   # precision, recall, and F1 at once

cross_val_score(DecisionTreeClassifier(random_state=3141),
                X, y, cv=10)                  # 10-fold cross-validation

# num_obs, num_pred stand in for observed and predicted values of a number.
root_mean_squared_error(num_obs, num_pred)  # RMSE for a numeric outcome
r2_score(num_obs, num_pred)                 # R squared as 1 - SSres/SStot
```